

DEV2 – JAVL – Laboratoire Java**TD 08 – Encapsulation****Résumé**

Ce TD poursuit notre étude de Java en introduisant l'encapsulation et le passage d'argument à une méthode.

Table des matières

1	L'encapsulation	2
2	La visibilité	4
3	Passage d'argument à une méthode	4
4	Retour sur l'annuaire	8

1 L'encapsulation

Reprenons notre classe `Dog` écrite dans les TD précédents, elle devrait ressembler à ceci (le code des méthodes a été omis) :

```
/**
 * Represents a dog inside the program
 * @author g12345
 */
public class Dog {
    String name;
    int weight;
    boolean hungry = false;
    boolean injured = false;

    Dog (String name, int weight){
        this.name = name;
        this.weight = weight;
    }
    // Méthodes de la classe Dog ...
}
```

Le TD02 a introduit les notions d'**accesseur** (getter) et de **mutateur** (setter) pour accéder aux valeurs ou assigner de nouvelles valeurs aux attributs.

Cependant, en l'état, il est tout à fait possible d'accéder et de modifier la valeurs des attributs à partir de notre classe `Main` sans utiliser ces getter/setter :

```
public class Main{
    public static void main (String[] args){
        Dog médor = new Dog("Médor",8);

        // Les modifications des attributs ci-dessous n' ont pas beaucoup de sens ...
        médor.name = "";
        médor.weight = -234;
    }
}
```

A l'évidence, permettre à du code manipulant un objet chien de corrompre les données des attributs de la sorte n'est pas souhaitable, ce qui nous amène au concept d'encapsulation (voir slides du cours) :

Orienté objet - Encapsulation

Premier principe de l'orienté objet : l'**encapsulation**

*La cohérence de l'objet
est assurée par la classe*

- les attributs sont **privés**
- les méthodes permettant de modifier l'état de l'objet sont **publiques** ; elles s'assurent que l'objet reste cohérent

Rendez les différents attributs de la classe `Dog` privé en ajoutant simplement le mot clé `private` dans leur déclaration :

```
private String name;
private int weight;
private boolean hungry = false;
private boolean injured = false;
```

Une tentative de modification d'un attribut privé depuis une autre classe que `Dog` est alors immédiatement détectée par IntelliJ comme étant illégale (tenter de compiler le code produira une erreur) :



La modification de l'attribut `weight` ne doit pouvoir se faire que via les méthodes publiques de la classes, dont on peut écrire le code de façon à ce qu'il vérifie la validité de la modification. Modifiez par exemple le setter de `weight` ainsi¹ :

```
public final void setWeight(int weight){
    if (weight <= 0){
        throw new IllegalArgumentException("Le poids doit être un entier positif !");
    }
    this.weight = weight;
}
```

Adaptez aussi le constructeur de façon à refuser les valeurs invalides du paramètre :

```
public Dog (String name, int weight){
    this.name = name;
    // La ligne ci-dessous déclenchera une IllegalArgumentException si weight <= 0
    setWeight(weight);
}
```

Exercice 1

L'encapsulation (classe `Dog`)

Adaptez le constructeur de l'attribut `name` de la classe `Dog` : le nom d'un chien doit toujours comporter entre 2 et 30 caractères. Une `IllegalArgumentException` doit être déclenchée le cas échéant.

Exercice 2

L'encapsulation (classe `Contact`)

Adaptez le constructeur et la déclaration des attributs de la classe `Contact` écrite au TD02 de façon à respecter l'encapsulation :

- ▷ les attributs `name`, `firstName`, `adresse`, `phone` deviennent privés. Ecrivez (ou générez automatiquement au moyen d'IntelliJ) les getters et setters publiques pour ces attributs.
- ▷ les noms, prénoms, adresse doivent comporter au moins deux caractères. Le numéro de téléphone doit être un entier positif comportant au moins 4 chiffres. Adaptez les setter et le constructeur de la classe de façon à satisfaire ces exigences, déclenchez des `IllegalArgumentException` lorsque nécessaire.

1. Notez le mot `final` ajouté dans la signature. Il a une raison d'exister que ne nous pourrions expliquer qu'au moment de parler d'héritage. D'ici là, vous pouvez le mettre ou vous pouvez l'omettre.

2 La visibilité

Nous venons de voir comment modifier la **visibilité** d'un attribut de façon à le rendre privé : l'attribut ne peut-être accédé directement que via le code de la classe. Le langage Java offre plusieurs options de visibilité :

La visibilité en Java

En Java, un attribut ou une méthode d'une classe peuvent-être :

1. **Privé** : l'attribut n'est accessible qu'au sein même de la classe, utiliser le mot clé `private`,
2. **Public** : l'attribut est accessible partout (à partir de n'importe quelle autre classe), utiliser le mot clé `public`,
3. **Package** : l'attribut est accessible à partir d'une classe située dans le même package, n'utiliser aucun mot clé,
4. **Protected** : ce type de visibilité est lié à l'héritage (nous en parlerons plus loin dans le cours de Dev2).

Le code suivant déclare par exemple un attribut `name` privé, un attribut `weight` public et un attribut `hungry` package (remarquez l'absence de mot clé devant le type de donnée, `boolean`, de cet attribut) :

```
public String name;  
private int weight;  
boolean hungry;
```

Bien que cela soit permis par le langage, c'est (en général) une **mauvaise idée de déclarer les attributs public en Java** : cela viole le principe d'encapsulation. On s'en tient généralement à ce principe :

1. Les attributs sont déclarés `private`,
2. Des méthodes `public` permettent d'y accéder.

Déclarer des attributs public ou package ne se justifie que dans de rares cas, que vous apprendrez à reconnaître dans la suite de votre apprentissage du langage.

3 Passage d'argument à une méthode

Nous venons de voir comment modifier un objet dans la méthode `main` de notre programme tout en respectant l'encapsulation. Mais qu'en est-il si nous passons cet objet en paramètre à une méthode ? De façon générale, une méthode peut-elle modifier les valeurs des variables qui lui sont passées en paramètre ?

Exercice 3

Le passage de paramètre

Exécutez le code suivant :

```
public class Main{  
    public static void main (String[] args){  
        int data = 5;  
        System.out.println("La variable data vaut " + data);  
        modify(data);  
        System.out.println("La variable data vaut à présent " + data);  
    }  
  
    public static void modify (int i){
```

```

    i = 10;
  }
}

```

Qu'affiche le programme lors de son exécution ?

Exercice 4

Le passage de paramètre (suite)

Exécutez maintenant le code suivant :

```

public class Main{
    public static void main (String[] args){
        Dog médor = new Dog ("Médor", 5);
        System.out.println("Médor pèse " + médor.getWeight());
        modify(médor);
        System.out.println("Médor pèse à présent " + médor.getWeight());
    }

    public static void modify (Dog myDog){
        myDog.setWeight(10);
    }
}

```

Qu'affiche ce programme lors de son exécution ?

Comme vous pouvez le constater, il est possible de modifier un objet dans une méthode alors que c'est impossible pour une variable appartenant à un type de donnée primitif.

Types primitifs et types référence en Java

Certains types de données ne sont pas implémentés comme des objets en Java, ce sont les **types primitifs** : byte, short, int, long, double, float, boolean et char ^a. Tous les autres types de données sont considérés comme des objets et appelés des **types référence**.

^a. Remarquez que le nom d'un type primitif commence toujours par une minuscule (contrairement aux types références), ceci permet de facilement les reconnaître !

On peut voir une variable d'un type primitif comme un emplacement mémoire contenant directement la valeur qui lui est assignée. Par exemple :

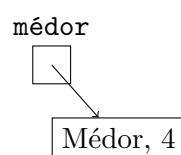
```
int i = 7;
```

peut se représenter au moyen du schéma :



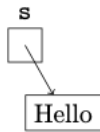
Par contre, un variable d'un type référence contient une **référence** vers l'objet, qui est stocké à un certain endroit dans la mémoire. On peut représenter ceci au moyen d'une flèche, comme illustré sur l'exemple ci-dessous :

```
Dog médor = new Dog("Médor",4);
```



Attention : tout type de donnée autre que ceux cités dans l'encadré ci-dessus sont des types références en Java (que ce type soit lié à une classe écrite par le/la développeur/euse ou non), y compris les String (chaînes de caractères) et les tableaux :

```
String s = "Hello";
```



Lors du passage de paramètre à une méthode, la valeur de la variable est copiée dans le paramètre de la méthode (qui est une variable locale à la méthode, et détruite à la fin de son exécution). Dans le cas d'une variable de type référence, la méthode a donc accès à une référence vers l'objet ce qui explique que l'objet soit modifiable dans la méthode.

Exercice 5

Le passage de paramètre (suite et fin)

Exécutez finalement le code suivant :

```
public class Main{
    public static void main (String[] args){
        Dog médor = new Dog ("Médor", 5);
        System.out.println("Médor pèse " + médor.getWeight());
        modify(médor);
        System.out.println("Médor pèse à présent " + médor.getWeight());
    }

    public static void modify (Dog myDog){
        myDog = new Dog("Milou",3);
    }
}
```

Qu'affiche ce programme lors de son exécution ? Faites un schéma mémoire (c'est-à-dire un dessin représentant les différentes zones de mémoire avant, pendant et après l'exécution de la méthode) permettant d'expliquer le résultat.

Exercice 6 La nourriture

Dans votre classe `Main`, ajoutez une méthode

```
public static void feed (Dog myDog, int minWeight)
```

qui nourrit le chien si son poids est inférieur à `minWeight`. Pour nourrir un chien, veuillez modifier la méthode `eat` de la classe `Dog` écrite précédemment de façon à ce qu'un appel à `eat` augmente le poids du chien de 1kg. La méthode `feed` doit faire appel à `eat` si nécessaire, c'est-à-dire si le chien possède un poids inférieur à `minWeight` (sinon la méthode ne fait rien).

Remarquez qu'un objet peut lui même contenir des références vers d'autres objets, ce sera notamment le cas avec les listes d'objets. Dans le code ci-dessous, `dogs` référence un objet (de type `ArrayList`) qui référence lui-même deux objets (de type `Dog`) :

```
List<Dog> dogs = new ArrayList<>();  
dogs.add(new Dog("Médor", 5));  
dogs.add(new Dog("Milou", 4));
```

Exercice 7 La liste d'objets

Dessiner un schéma mémoire de la liste `dogs` définie dans le code java ci-dessus.

Exercice 8 La liste d'objets (suite)

Considérons le code Java ci-dessous :

```
List<Dog> dogs1 = new ArrayList<>();  
List<Dog> dogs2 = new ArrayList<>();  
  
// Construction de la liste dogs1 ici ...  
dogs1.add(new Dog("Médor", 5));  
dogs1.add(new Dog("Fido", 7));  
dogs1.add(new Dog("Rex", 10));  
dogs1.add(new Dog("Luna", 6));  
dogs1.add(new Dog("Bella", 8));  
dogs1.add(new Dog("Max", 12));  
dogs1.add(new Dog("Coco", 9));  
  
// Construction de dogs2 ici ...  
dogs2.add(new Dog("Milou", 8));  
dogs2.add(dogs1.get(0));  
dogs2.get(1).setWeight(230);  
  
// Qu'est-ce qui sera affiché ici ?  
System.out.println(dogs1.get(0).getWeight());
```

Que va afficher à l'écran la dernière ligne de code ? Expliquer le résultat au moyen d'un schéma mémoire.

Exercice 9 La nourriture (suite)

Modifiez votre méthode `feed` de façon à prendre en paramètre une liste de chiens (plutôt qu'un unique chien) :

```
public static void feed (List<Dog> dogs, int minWeight)
```

La méthode nourrit tous les chiens de la liste dont le poids est inférieur à `minWeight`.

Testez votre méthode à partir de la méthode `Main` en créant une liste de chiens de différents poids et en appelant la méthode `feed` sur cette liste. Vérifiez que la liste est bien mise à jour comme on s'y attend.

4 Retour sur l'annuaire

Reprenons à présent la classe `PhoneBook` écrite dans le TD03.

Exercice 10 L'encapsulation (classe `PhoneBook`)

Adaptez le code de la classe `PhoneBook` de façon à respecter l'encapsulation (à vous de voir quoi changer exactement).

Exercice 11 Modifier un contact

Ajoutez une méthode publique :

```
public void update (int numContact, String adresse, int phone)
```

qui modifie l'adresse du contact numéro `numContact` avec l'adresse et le numéro de téléphone fourni en paramètre.

Exercice 12 Fusionner deux annuaires

Ajoutez finalement une méthode publique :

```
public void merge (PhoneBook pb)
```

qui fusionne l'annuaire `pb` à l'annuaire courant : tous les contacts de ce dernier sont ajoutés à la fin de la liste de contacts de l'annuaire courant (l'annuaire `pb` n'est pas modifié).